Scheduling jobs with quadratic load cost

Using dynamic programming (failed attempt)

Klevis Imeri

The input to the problem:

$$\mathcal{I} := \left\{ \begin{array}{ll} N := \{\text{jobs}\}, & |N| = n, \\ P[i] := \text{processing time of job } i, & |P| = n, \quad P[i] > 0, \\ M := \{\text{machines}\}, & |M| = m. \end{array} \right\}.$$

The exercise allows $P[i] \geq 0$, but zero-processing-time jobs do not affect the cost. We set them aside and assign them to any machine at the end, so we assume $P[i] > 0$. If all jobs have processing time zero, any schedule is optimal.

The output is a schedule $\mathcal{S} = (\mathcal{S}[k])_{k \in M}$, where

$$\mathcal{S}[k] := \{i \in N \mid \text{job } i \text{ is scheduled on machine } k\}.$$

$$\begin{aligned} \text{feasible}(\mathcal{S}) := & \bigcup_{k \in M} \mathcal{S}[k] = N \\ & \wedge \ \forall k, \ell \in M : \quad k \neq \ell \implies \mathcal{S}[k] \cap \mathcal{S}[\ell] = \varnothing. \end{aligned}$$

The cost function is

$$L_k := \sum_{i \in \mathcal{S}[k]} P[i], \qquad f(\mathcal{S}) := \sum_{k=1}^{m} \left(\sum_{i \in \mathcal{S}[k]} P[i] \right)^2 = \sum_{k=1}^{m} L_k^2.$$

We seek

$$\text{OPT}(\mathcal{I}) \in \underset{\mathcal{S}: \text{ feasible}(\mathcal{S})}{\arg\min} \ f(\mathcal{S}).$$

Goal 1. For a fixed constant $C \in \mathbb{N}$, assume

$$\big|\{P[i] \mid i \in N\}\big| \leq C.$$

Our goal is to design an algorithm $\mathcal{A}_C$ with $\mathcal{S}_{\mathcal{A}_C} = \mathcal{A}_C(\mathcal{I})$ such that:

1. $\mathcal{A}_C$ terminates.
2. $\text{feasible}(\mathcal{S}_{\mathcal{A}_C})$.
3. $\text{time}_{\mathcal{A}_C}(\mathcal{I}) \leq \text{poly}(|\mathcal{I}|)$.
4. $f(\mathcal{S}_{\mathcal{A}_C}) = f(\text{OPT}(\mathcal{I}))$.

One may think this problem is very similar to the exact bin packing algorithm in the lecture, where we assumed:

1. A constant number of different sizes.

2. Every item has size at least ε, for a fixed $\varepsilon > 0$.

However, here we have at most C different processing times and $P[i] > 0$, but no fixed positive lower bound, so the second assumption does not hold in general ($P[i]$ can be very small, allowing many jobs on one machine, so we cannot bound their number by a constant as in the lecture). Moreover, our machines have no capacity bound, so even a positive lower bound on processing times would not give a constant bound on the number of jobs per machine.

A machine can contain at most n jobs. Thus, by the stars and bars method, the number of possible configurations is at most

$$R = \binom{n+C}{C} = O(n^C).$$

This is polynomial for fixed C, but R is no longer constant. This is a problem since, similarly to the lecture, at most n nonempty machines are needed, so the total number of combinations of machine configurations is at most

$$\binom{n+R}{R} \leq (n+R)^R = n^{O(n^C)}.$$

This bound is not polynomial, so the lecture's enumeration does not give a polynomial running-time guarantee here.

Let us therefore analyze some properties of how squares behave. For a fixed total load, the sum of squared loads is minimized when all machines have the same load

$$\overline{L} := \frac{\sum_{i \in N} P[i]}{m}.$$

This is quite intuitive, but to put it precisely, write

$$L_k = \overline{L} - (\overline{L} - L_k).$$

The sum of all deviations from the average is zero:

$$\sum_{k=1}^{m} (L_k - \overline{L}) = \sum_{i \in N} P[i] - m\overline{L} = 0.$$

Thus,

$$\sum_{k=1}^{m} L_k^2 = m\overline{L}^2 + 2\overline{L} \sum_{k=1}^{m} (L_k - \overline{L}) + \sum_{k=1}^{m} (L_k - \overline{L})^2 = m\overline{L}^2 + \sum_{k=1}^{m} (L_k - \overline{L})^2 \geq m\overline{L}^2, \qquad (1)$$

with equality exactly when every $L_k = \overline{L}$. Since jobs cannot be split, this may not be possible. We therefore want the machine loads to be close to this average, in the sense of minimizing

$$\sum_{k=1}^{m} L_k^2 = m\overline{L}^2 + \sum_{k=1}^{m} (L_k - \overline{L})^2.$$

Intuitively, our lower bound is optimal in the continuous (fractional) setting, but our choices are discrete. We are therefore looking for a discrete schedule whose cost is as close as possible to this fractional optimum. This resembles the distinction between LP and ILP, or between fractional and 0/1 knapsack from the lecture, where we used dynamic programming.

It is not clear that going from LP to ILP would give us an exact solutoin. Let us also try to construct a dynamic programming algorithm for this problem.

To do that, we need a choice to iterate over and a subproblem whose saved solution lets us reconstruct the current solution. We know from above that there are at most R possible configuratoins for one machine. Let our choice be the configuration assigned to the last machine. The subproblem then has the remaining job-count vector minus the chosen configuration, and one fewer machine. Thus, we add one machine at each stage and consider all possible configurations for it.

As part of our dynamic programming algorithm, we construct P' from P by keeping its distinct processing times:

$$P' = (p'_1, \ldots, p'_c), \qquad \{p'_1, \ldots, p'_c\} = \{P[i] \mid i \in N\}, \qquad c \leq C.$$

Jobs with the same processing time belong to the same type.

Given our reasoning above, our recurrence is

$$\mathrm{DP}[k, U] = \min_{0 \leq I \leq U} \left\{ \mathrm{DP}[k-1, U-I] + (I^\mathsf{T} P')^2 \right\}.$$

$$\begin{aligned} k &: \text{ number of machines,} \\ P' = (p'_1, \ldots, p'_c) &: \text{ vector of processing times, one per job type,} \\ U = (u_1, \ldots, u_c) &: \text{ vector counting jobs of each type to schedule,} \\ I = (i_1, \ldots, i_c) &: \text{ vector counting jobs of each type on the last machine,} \\ \mathrm{DP}[k, U] &: \text{ minimum cost of scheduling } U \text{ on } k \text{ machines.} \end{aligned}$$

The vector inequalities are componentwise. We use the base cases

$$\mathrm{DP}[0, \mathbf{0}] = 0, \qquad \mathrm{DP}[0, U] = +\infty \quad (U \neq 0).$$

For reconstuction, we save a minimizing configuration I^* in $\mathrm{CH}[k][U]$ for each state. Below, we give the exact algorithm.

Now we reason about our four goals and prove that our algorithm has these properties.

1. Termination. There is no recursion, and all loops and configuration choices range over finite sets. Therefore, the algorithm terminates.
2. Feasibility.
   (a) During reconstruction, we assign the jobs specified by I to machine k and update $U \leftarrow U - I$. The reconstruction must reach $\mathrm{DP}[0][\mathbf{0}]$ because reaching $\mathrm{DP}[0][U]$ with $U \neq \mathbf{0}$ would give cost $+\infty$. Our algorithm would only choose an infinite cost if every candidate in the arg minimum had infinite cost. This cannot happen because we consider all $I \leq U$, including $I = U$, which gives

   $$\mathrm{DP}[k-1][\mathbf{0}] + (U^\mathsf{T} P')^2 = 0 + (U^\mathsf{T} P')^2 < +\infty.$$

   Thus, no jobs remain unassigned, and

   $$\bigcup_{k \in M} \mathcal{S}[k] = N.$$

   (b) Each machine receives only unassigned jobs. Therefore,

   $$\forall k, \ell \in M: \quad k \neq \ell \implies \mathcal{S}[k] \cap \mathcal{S}[\ell] = \varnothing.$$

Algorithm 1 *Exact-Scheduling*$_C(\mathcal{I})$

Construct P' and U^{all}, the counts of jobs of each type
Let DP, CH
DP[0][$\mathbf{0}$] $\leftarrow 0$
for each job-count vector $\mathbf{0} \neq U \leq U^{\text{all}}$ **do**
  DP[0][U] $\leftarrow +\infty$
end for
for $k = 1, \ldots, m$ **do**
  for each job-count vector $\mathbf{0} \leq U \leq U^{\text{all}}$ **do**
    $I^* \leftarrow \text{CH}[k][U] \leftarrow \arg\min_{I \leq U} \left\{\text{DP}[k-1][U-I] + (I^\mathsf{T} P')^2\right\}$
    $\text{DP}[k][U] \leftarrow \text{DP}[k-1][U-I^*] + ((I^*)^\mathsf{T} P')^2$
  end for
end for
Let $\mathcal{S}$; $U \leftarrow U^{\text{all}}$
for $k = m, m-1, \ldots, 1$ **do**
  $I \leftarrow \text{CH}[k][U]$
  $\mathcal{S}[k] \leftarrow$ any unassigned jobs matching configuration I
  $U \leftarrow U - I$
end for
return $\mathcal{S}$

Both conditions hold, so feasible($\mathcal{S}_{\mathcal{A}_C}$).

3. Running time. Constructing P' and U^{all} takes $O(nC) = O(n)$ time for fixed C. Each component of a job-count vector has at most $n+1$ possible values, including zero, and there are at most C components. Thus, there are at most $(n+1)^C$ job-count vectors. Using arrays, the table sizes, measured in stored numbers, satisfy

$$|\text{DP}| \leq (m+1)(n+1)^C, \qquad |\text{CH}| \leq C(m+1)(n+1)^C,$$

because each entry of CH stores a configuration vector. The base-case initialization fills at most $(n+1)^C$ states.
In the DP loops, we consider m machine counts and at most $(n+1)^C$ job-count vectors. For each state, the arg minimum considers at most $(n+1)^C$ configurations, each taking $O(C)$ time for vector operations, table access, and the scalar product. Thus, the DP computation takes

$$O\big(mC(n+1)^{2C}\big)\,.$$

Initializing reconstruction takes $O(C)$. Reconstruction takes $O(m(n+C)) = O(mn)$ for fixed C, by scanning the jobs for each machine and selecting the required unassigned jobs. Overall,

$$T = O\big(nC + (m+1)C(n+1)^C + mC(n+1)^{2C} + mn\big) = O\big(mC(n+1)^{2C}\big)\,,$$

which is poly($|\mathcal{I}|$) for fixed C.

4. Optimal solution. We have to prove that our solution is one of the optimal solutions. As usual with dynamic programming, we prove that the DP invariant holds. We prove by induction on k that, for every $\mathbf{0} \leq U \leq U^{\text{all}}$,

$$\text{DP}[k][U] = \min_{I_1 + \cdots + I_k = U} \sum_{\ell=1}^{k} (I_\ell^\mathsf{T} P')^2,$$

and, for $k \geq 1$, $\mathrm{CH}[k][U]$ stores a last-machine configuration attaining this minimum together with an optimal schedule for the remaining jobs.

Base case. $\mathrm{DP}[0][\mathbf{0}] = 0$ is optimal because there are no jobs, so no cost. For $U \neq \mathbf{0}$, no schedule on zero machines exists, so $\mathrm{DP}[0][U] = +\infty$.

Inductive step. Assume the invariant holds for $k-1$ and every job-count vector. Any schedule on k machines assigns some configuration $I \leq U$ to its last machine, leaving $U - I$ for the other $k-1$ machines. By the induction hypothesis, their minimum cost is $\mathrm{DP}[k-1][U-I]$. Conversely, combining such an optimal schedule with configuration I gives cost

$$\mathrm{DP}[k-1][U-I] + (I^{\mathsf{T}} P')^2.$$

Our algorithm considers every possible $I \leq U$ and selects a minimum, so $\mathrm{DP}[k][U]$ is optimal and $\mathrm{CH}[k][U]$ stores a minimizing last-machine configuration. Thus, the invariant holds.

The reconstruction follows these stored minimizing configurations, so its total cost is $\mathrm{DP}[m][U^{\mathrm{all}}]$. Therefore,

$$f(\mathcal{S}_{\mathcal{A}_C}) = \mathrm{DP}[m][U^{\mathrm{all}}] = f(\mathrm{OPT}(\mathcal{I})).$$

□

Goal 2. Our goal is to design an algorithm $\mathcal{A}_\varepsilon$ with $\mathcal{S}_{\mathcal{A}_\varepsilon} = \mathcal{A}_\varepsilon(\mathcal{I})$, such that:

1. $\mathcal{A}_\varepsilon$ terminates.
2. $\mathrm{feasible}(\mathcal{S}_{\mathcal{A}_\varepsilon})$.
3. $\mathrm{time}_{\mathcal{A}_\varepsilon}(\mathcal{I}) \leq \mathrm{poly}(|\mathcal{I}|)$ for fixed ε.
4. $f(\mathcal{S}_{\mathcal{A}_\varepsilon}) \leq (1+\varepsilon) f(\mathrm{OPT}(\mathcal{I})),\ \forall \varepsilon > 0$ ($\mathcal{A}_\varepsilon$ is a PTAS).

As we did in class with the knapsack FPTAS, we will try rounding the processing times.

Algorithm 2 *PTAS-Scheduling*$(\mathcal{I}, \varepsilon)$

Choose $K \in (0, ?]$
for $i \in N$ **do**
  $\widehat{P}[i] \leftarrow K \left\lfloor \dfrac{P[i]}{K} \right\rfloor$
end for
$\widehat{\mathcal{I}} \leftarrow \{N, \widehat{P}, M\}$
$C \leftarrow \left|\{\widehat{P}[i] \mid i \in N\}\right|$
return *Exact-Scheduling*$_C(\widehat{\mathcal{I}})$

Notice the choice of K. In the lecture, for knapsack, we chose

$$K \leftarrow \max\left\{1, \left\lfloor \frac{\varepsilon}{n} p_{\max} \right\rfloor\right\}.$$

Here, we cannot impose $K \geq 1$ because all processing times could be smaller than 1, in which case rounding would make them all zero. We must therefore allow K to be smaller than 1, while keeping $K > 0$ because we divide by it. The symbol ? means that we have not yet chosen the upper bound on K because we will determine it during the proof to meet the PTAS requirements.

Also, unlike knapsack from class, where we maximize profit, here we minimize the sum of squared machine loads. We use the standard inequality step, similar to knapsack in the lecture, where for maximization we had

$$c(\mathrm{OPT}) \geq c(\mathrm{OPT}') \geq c'(\mathrm{OPT}') \geq c'(\mathrm{OPT}).$$

Here, for minimization, we have

$$c'(\mathrm{OPT}') \le c'(\mathrm{OPT}) \le c(\mathrm{OPT}) \le c(\mathrm{OPT}').$$

In our notation,

$$\widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) \le \widehat{f}(\mathrm{OPT}) \le f(\mathrm{OPT}) \le f(\mathcal{S}_{\mathcal{A}_\varepsilon}).$$

Here, $\widehat{f}$ is the cost computed using the rounded processing times. Either way, the idea is to bound the error by $\varepsilon f(\mathrm{OPT})$. Thus, we need to show

$$\underbrace{f(\mathcal{S}_{\mathcal{A}_\varepsilon}) - \widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) \le \varepsilon f(\mathrm{OPT})}_{\text{sufficient error bound}} \tag{2}$$

$$\iff \quad f(\mathcal{S}_{\mathcal{A}_\varepsilon}) \le \widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) + \varepsilon f(\mathrm{OPT}).$$

Since $\mathcal{S}_{\mathcal{A}_\varepsilon}$ minimizes the rounded cost,

$$\begin{aligned} f(\mathcal{S}_{\mathcal{A}_\varepsilon}) &\le \widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) + \varepsilon f(\mathrm{OPT}) \\ &\le \widehat{f}(\mathrm{OPT}) + \varepsilon f(\mathrm{OPT}) \\ &\le f(\mathrm{OPT}) + \varepsilon f(\mathrm{OPT}) \\ &= (1+\varepsilon) f(\mathrm{OPT}). \end{aligned}$$

It is worth mentioning that, since we round down, $\widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) \le f(\mathcal{S}_{\mathcal{A}_\varepsilon}) \implies f(\mathcal{S}_{\mathcal{A}_\varepsilon}) - \widehat{f}(\mathcal{S}_{\mathcal{A}_\varepsilon}) \ge 0$.

So now, the only thing left to prove for the approximation guarantee is the sufficient error bound in equation (2).

For a generic $K > 0$, write $S = \mathcal{S}_{\mathcal{A}_\varepsilon}$. The load error on each machine satisfies

$$0 \le L_k - \widehat{L}_k = \sum_{i \in S[k]} \left(P[i] - K \left\lfloor \frac{P[i]}{K} \right\rfloor \right) \le |S[k]| K \le nK.$$

Thus,

$$\begin{aligned} f(S) - \widehat{f}(S) &= \sum_{k=1}^{m} (L_k - \widehat{L}_k)(L_k + \widehat{L}_k) \\ &\le nK \sum_{k=1}^{m} (L_k + \widehat{L}_k) \\ &\le 2nK \sum_{k=1}^{m} L_k = 2nK \sum_{i \in N} P[i]. \end{aligned}$$

We will choose K so that this bound is at most $\varepsilon f(\mathrm{OPT})$. From equation (1) above, we know

$$f(\mathrm{OPT}) \ge m\overline{L}^2.$$

Thus, for $\overline{L} > 0$, a sufficient choice is

$$2nKm\overline{L} \le \varepsilon m\overline{L}^2 \iff K \le \frac{\varepsilon \overline{L}}{2n}.$$

Taking $K = \frac{\varepsilon \overline{L}}{2n}$ therefore gives the required approximation guarantee.

The problem is the running time. With $P_{\max} := \max_{i \in N} P[i] \le m\overline{L}$, the number of rounded processing-time types satisfies

$$C \le 1 + \left\lfloor \frac{P_{\max}}{K} \right\rfloor \le 1 + \frac{P_{\max}}{K} \le 1 + \frac{m\overline{L}}{\varepsilon \overline{L}/(2n)} = 1 + \frac{2nm}{\varepsilon}.$$

This bound depends on n and m, even for fixed ε. Our exact algorithm has running time $O(mC(n+1)^{2C})$, which is polynomail only for fixed C. Even if we could improve the bound to $P_{\max} \leq a\overline{L}$ for a constant a, we would still have the n in the denominator of K, giving

$$C \leq 1 + \frac{a\overline{L}}{\varepsilon\overline{L}/(2n)} = 1 + \frac{2an}{\varepsilon}.$$

So the dependence on n would still cause problems. Thus, this analysis gives the approximatoin guarantee but does not give a polynomial running-time guarantee. We need a better rounding or grouping method that controls the error while keeping the number of types dependent only on ε.

Let us step back and see what the bigger problem could be. We allowed each of the n jobs to contribute up to K to the rounding error, giving a load error bounded by nK, as in knapsack from the lecture. Here, however, our DP has C in the exponent. In other words, controlling the acumulated error forced too many rounded types.

The first idea is that, because we want to minimize $\sum_{k=1}^{m}(L_k - \overline{L})^2$, if $P[i] \geq \overline{L}$, we can assign job i to a machine on its own in an optimal schedule. By intuition, we would rather use the other jobs to bring less loaded machines closer to $\overline{L}$ than add them to this machine and move its load even farther above $\overline{L}$.

For the formal proof, suppose an optimal schedule puts other jobs of total load $x > 0$ on the machine containing job i. This moves its load from $P[i] \geq \overline{L}$ to $P[i] + x$, farther above the average. Since this machine is above the average, another machine has load $b < \overline{L}$. We can instead put those other jobs on that less loaded machine. The total squared distance from the average decreases, because

$$\begin{aligned}&(P[i] + x - \overline{L})^2 + (b - \overline{L})^2 \\ &\quad - \big((P[i] - \overline{L})^2 + (b + x - \overline{L})^2\big) = 2\underbrace{x}_{>0}\ \underbrace{(P[i] - b)}_{>0 \text{ since } P[i] \geq \overline{L} > b} > 0.\end{aligned}$$

Thus, using the other jobs on the less loaded machine gives a better schedule, contradicting optimality.

By setting these large jobs aside on their own machines and recomputing $\overline{L}$ after each removal, the remaining instance has $P_{\max} \leq \overline{L}$. This removes the factor m from our bound on C, giving $C \leq 1 + 2n/\varepsilon$. However, we still have the dependence on n to remove.